

AUTD3 触覚ボード  
リファレンスマニュアル  
Python 編

| 版   | 日付         | 内容              |
|-----|------------|-----------------|
| 1.0 | 2026/03/19 | 初版              |
| 1.1 | 2026/04/09 | EtherCrab 関連を追加 |
|     |            |                 |
|     |            |                 |

## 目次

|                                                   |    |
|---------------------------------------------------|----|
| 1. 概要 .....                                       | 2  |
| 2. インストール .....                                   | 2  |
| 2.1. パッケージ一覧及び共通事項 .....                          | 2  |
| 2.2. インストール .....                                 | 3  |
| 3. 制御手順 .....                                     | 4  |
| 3.1. 基本構造 .....                                   | 4  |
| 3.2. モジュール構成とデータの流れ .....                         | 5  |
| 3.3. 操作フロー .....                                  | 6  |
| 4. クラス/メソッド詳細 .....                               | 7  |
| 4.1. クラス/メソッド一覧 .....                             | 7  |
| 4.2. Controller クラス .....                         | 10 |
| 4.3. AUTD3 クラス .....                              | 14 |
| 4.4. Geometry クラス/Device クラス/Transducer クラス ..... | 15 |
| 4.5. TwinCAT クラス .....                            | 18 |
| 4.6. EtherCrab クラス/EtherCrabOption クラス .....      | 20 |
| 4.7. Remote クラス/RemoteOption クラス .....            | 22 |
| 4.8. Silencer クラス .....                           | 23 |
| 4.9. Focus クラス/FocusOption クラス .....              | 24 |
| 4.10. Plane クラス/PlaneOption クラス .....             | 25 |
| 4.11. Bessel クラス/BesselOption クラス .....           | 26 |
| 4.12. FociSTM クラス .....                           | 27 |
| 4.13. GainSTM クラス/GainSTMOption クラス .....         | 28 |
| 4.14. Null クラス .....                              | 29 |
| 4.15. Sine クラス/SineOption クラス .....               | 30 |
| 4.16. Square クラス/SquareOption クラス .....           | 31 |
| 4.17. Static クラス .....                            | 32 |
| 4.18. EtherCrab クラス/EtherCrabOption クラス .....     | 33 |

## 1. 概要

本書は、(空中触覚用超音波フェーズドアレイデバイス(以下、AUTD3 デバイス)を Python を用いて動作させる際の手順及び、ライブラリ仕様について記述する。

## 2. インストール

### 2.1. パッケージ一覧及び共通事項

- Python 用の AUTD3 デバイスライブラリを使用する際に必要となる Python パッケージを下記に示す。

| パッケージ名                 | 内容等                                   |
|------------------------|---------------------------------------|
| numpy                  | 学術計算用パッケージ                            |
| pyautd3                | AUTD3 デバイスを制御するライブラリ本体                |
| pyautd3_link_ethercrab | EtherCrab 経由で AUTD3 デバイスと通信するリンクモジュール |

- 「pyautd3\_link\_ethercrab」は EtherCrab 経由で AUTD3 デバイスと通信する際にのみ必要となる。
- 上記パッケージは PyPI(Python Package Index)に登録されており、pip 機能にてインストールできる。  
(pip 機能が利用できる様になっている必要がある)
- システムが仮想環境での利用を要求している場合(PEP 668)、  
事前に仮想環境を作成し、仮想環境に切り替えた状態でインストール/実行を行う必要がある。

## 2.2. インストール

- インターネットに接続されている環境にて、下記コマンドにてインストールを行う。

```
pip install pyautd3 pyautd3_link_ethercrab
```

または

```
python3 -m pip install pyautd3 pyautd3_link_ethercrab
```

- オフライン環境でインストールする場合、下記手順にてインストールを行う。

### ① ダウンロード

インターネットが利用できる環境より、下記をダウンロードする。

#### pyautd3

ダウンロード先 : <https://pypi.org/project/pyautd3/#files>

ダウンロードファイル

Windows 用 : [pyautd3-38.0.1-py3-none-win\\_amd64.whl](#)

Linux 用 : [pyautd3-38.0.1-py3-none-manylinux1\\_x86\\_64.whl](#)

MacOS 用 : [pyautd3-38.0.1-py3-none-macosx\\_11\\_0\\_arm64.whl](#)

#### pyautd3\_link\_ethercrab

ダウンロード先 : <https://pypi.org/project/pyautd3-link-ethercrab/#files>

ダウンロードファイル

Windows 用 : [pyautd3\\_link\\_ethercrab-38.0.1-py3-none-win\\_amd64.whl](#)

Linux 用 : [pyautd3\\_link\\_ethercrab-38.0.1-py3-none-manylinux1\\_x86\\_64.whl](#)

MacOS 用 : [pyautd3\\_link\\_ethercrab-38.0.1-py3-none-macosx\\_11\\_0\\_arm64.whl](#)

#### numpy

ダウンロード先 : <https://pypi.org/project/pyautd3-link-ethercrab/#files>

ダウンロードファイル

各環境用をダウンロード

例) Ubuntu24.04-64bit 用 : [numpy-2.4.3-cp312-cp312-manylinux\\_2\\_27\\_x86\\_64.manylinux\\_2\\_28\\_x86\\_64.whl](#)

- ### ② ダウンロードしたファイルをインストール対象 PC にコピーし、一つのフォルダ下に格納する

### ③ インストール

下記コマンドでインストールする。

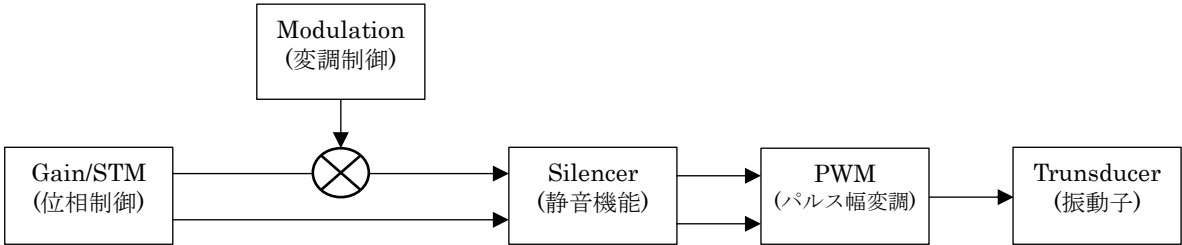
```
python3 -m pip install --no-index --find-links=フォルダパス numpy pyautd3 pyautd3_link_ethercrab
```

参考 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/getting\\_started/software.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/getting_started/software.html)

3. 制御手順

3.1. 基本構造

- AUTD3 デバイスが動作する際の概念図を下記に示す。

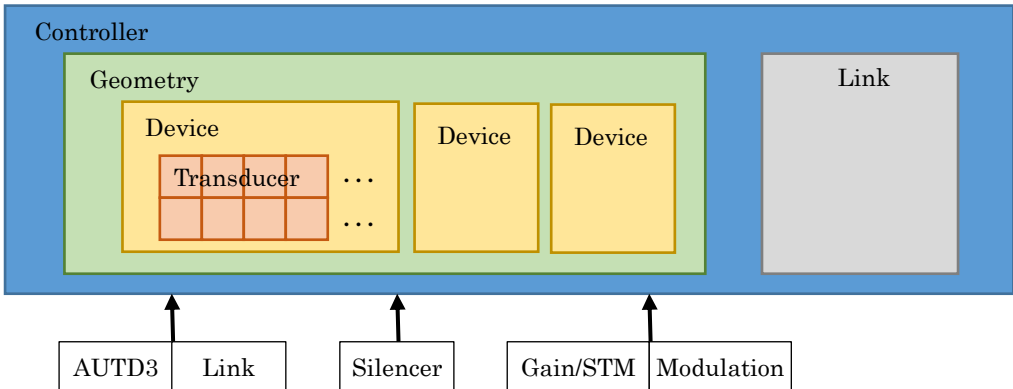


- Gain
- :
- 位相/振幅を管理する。
- STM
- :
- Spatio-Temporal Modulation(時空間変調)機能で、位相/振幅を時間列で管理する。
- Modulation
- :
- AM 変調を管理する。
- Silencer
- :
- 静音化处理を管理する。
- PWM
- :
- 上記データに基づき、各振動子を駆動させる電気信号を生成する。
- Trnsducer
- :
- 電気信号により超音波を発生させる。

- AUTD3 ライブラリの主な構成要素を下記に示す。  
(上記動作概念図中の要素と同名のクラスは、その要素の動作や状態等を管理する)

| コンポーネント              | 概要                                                                                          |
|----------------------|---------------------------------------------------------------------------------------------|
| Controller クラス       | AUTD3 デバイスに対する接続/指示送信などの操作を行う。                                                              |
| AUTD3 クラス            | 個々の AUTD3 デバイス情報を管理する。                                                                      |
| Geometry クラス         | AUTD3 デバイス群を管理する。<br>(Geometry クラスは Device クラスのコンテナであり、<br>Device クラスは Transducer クラスのコンテナ) |
| Link クラス※1           | AUTD3 デバイスとの通信を管理する。<br>TwinCAT 接続(TwinCAT クラス)、EtherCrab 接続(EtherCrab クラス)などがある。           |
| Gain クラス             | 位相/振幅を管理する。                                                                                 |
| STM クラス              | Spatio-Temporal Modulation(時空間変調)機能で、位相/振幅を時間列で管理する。                                        |
| Modulation クラス<br>※1 | AM 変調処理を管理する。<br>正弦波(Sine クラス)、矩形波(Square クラス)などがある。                                        |
| Silencer クラス         | 静音化处理を管理する。                                                                                 |

※1 実際にはこれらのクラスを継承したクラスを使用する。

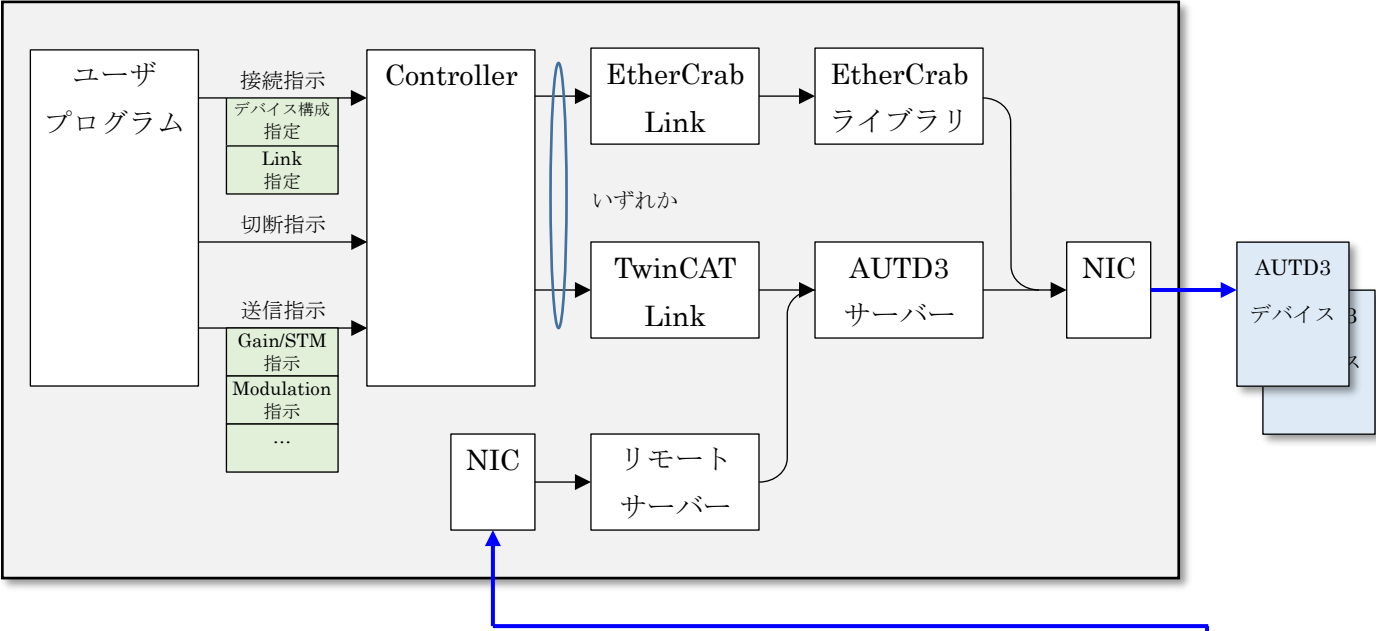


参考 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/tutorial/concept.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/tutorial/concept.html)

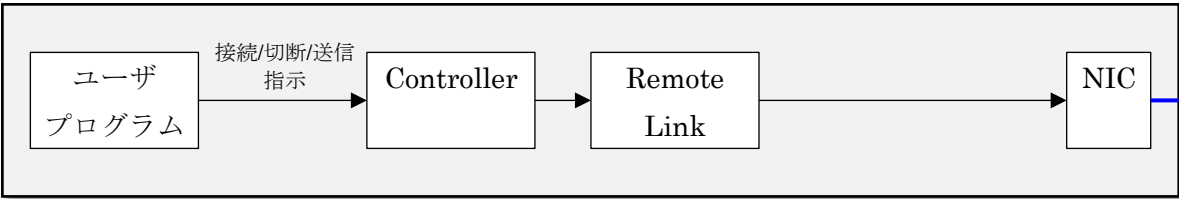
3.2. モジュール構成とデータの流れ

- 各モジュールの関連とデータの流れの概要を下記に示す。

AUTD3 デバイス(触覚ボード)が接続されている PC



AUTD3 デバイス(触覚ボード)が接続されていない PC

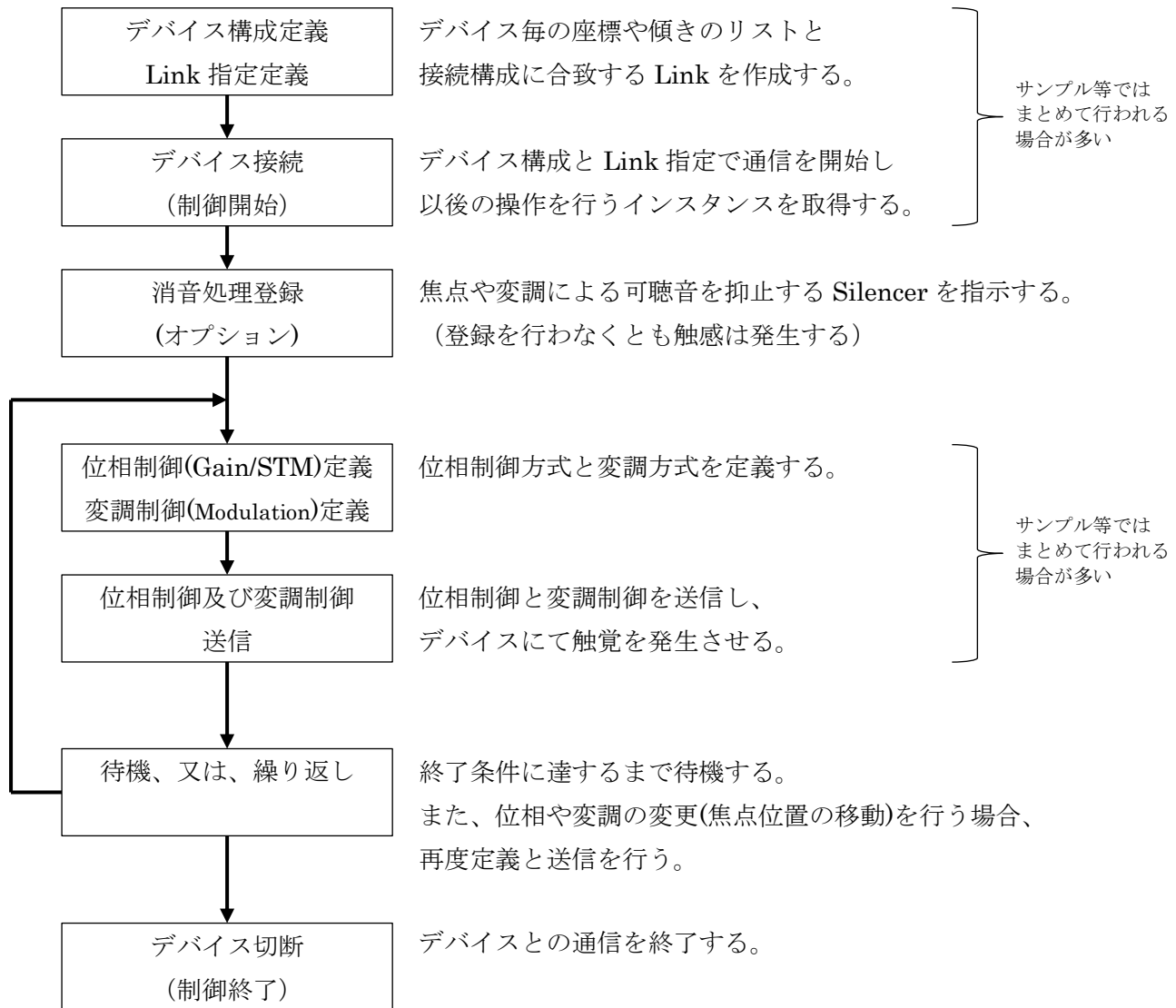


- ユーザプログラムコードは「Controller」に対してデバイスを動作させる各種操作を指示し、「Controller」は各種「Link」経由でデバイスとの通信を行う。
- Python では下記の「Link」が提供されている。

| Link 名         | 内容                                       |
|----------------|------------------------------------------|
| TwinCAT Link   | AUTD3 サーバー(TwinCAT ライブラリ)経由でデバイスと通信      |
| EtherCrab Link | EtherCrab ライブラリ(オープンソースライブラリ) 経由でデバイスと通信 |
| Remote Link    | リモートサーバ                                  |
- 「リモートサーバー」を用いる場合、デバイスが接続されていない PC から各種制御を行うこともできる。
- ユーザプログラムは位相制御(Grain/STM)や変調制御(Modulation)等の指示(Datagram を継承したクラス)を Controller 経由で AUTD3 デバイスに送信し、デバイスの動作を決定する。

### 3.3. 操作フロー

- ・ ユーザプログラムでボードを使用する際の手順を下記に示す。



- ・ 位相制御および変調制御を送信することにより触感が継続して生成される。  
送信処理はブロックせずにユーザコードに戻る為、必要に応じて焦点の変更や待機を行う。  
(ユーザアプリが終了すると触感生成も終了する為、触感生成を継続させるには待機等が必要)

## 4. クラス/メソッド詳細

### 4.1. クラス/メソッド一覧

- AUTD3 用ライブラリで提供されているクラス/メソッド/定数の一覧を下記に示す。

○pyautd3 パッケージ

| クラス           | メソッド(空白は定数等)     | 概要                            |
|---------------|------------------|-------------------------------|
| AUTD3         | __init__         | AUTD3 デバイス定義                  |
| Bessel        | __init__         | ベッセルビーム Gain 指示               |
| BesselOption  | __init__         | ベッセルビーム Gain オプション            |
| Clear         | __init__         |                               |
| Controller    | __init__         | 全体の動作管理                       |
|               | center           | 全振動子の中心座標を取得                  |
|               | close            | 切断                            |
|               | firmware_version | AUTD3 デバイスのバージョンを取得           |
|               | fpga_state       | FPGA 状態を取得                    |
|               | geometry         | AUTD3 デバイスの配置情報を取得            |
|               | link             | 使用される Link 指定を取得              |
|               | num_devices      | AUTD3 デバイスの数を取得               |
|               | num_transducers  | 全振動子の数を取得                     |
|               | open             | 接続                            |
|               | open_with_option | オプションを指定しての接続                 |
|               | send             | AUTD3 デバイスへの指示送信              |
|               | sender           | 送信時の設定を取得                     |
| Custom        | __init__         | 自由な音場 Gain 指示                 |
| ControlPoint  | __init__         | FociSTM での焦点座標情報              |
| ControlPoints | __init__         | FociSTM での焦点座標情報リスト           |
| DcSysTime     | __init__         | GPIO 出力項目 SysTimeEq で指定する時間   |
|               | sys_time         | 時刻の取得                         |
| Device        | __init__         | AUTD3 デバイス 1 枚の情報             |
|               | axial_direction  | AUTD3 デバイスの軸方向ベクトル (振動子が向く方向) |
|               | center           | 振動子の中心座標を取得                   |
|               | idx              | AUTD3 デバイスのインデックスを取得          |
|               | num_transducers  | AUTD3 デバイス内の振動子の数を取得          |
|               | rotation         | AUTD3 デバイスの回転                 |
|               | x_direction      | AUTD3 デバイスの x 方向ベクトル          |
|               | y_direction      | AUTD3 デバイスの y 方向ベクトル          |
| Drive         | __init__         |                               |
| Duration      | __init__         | 時間間隔の指定                       |
|               | as_micros        | μ 秒単位での時間間隔を取得                |
|               | as_millis        | ミリ秒単位での時間間隔を取得                |
|               | as_nanos         | ナノ秒単位での時間間隔を取得                |
|               | as_secs          | 秒単位での時間間隔を取得                  |
|               | from_micros      | μ 秒単位での時間間隔指定                 |
|               | from_millis      | ミリ秒単位での時間間隔指定                 |
|               | from_nanos       | ナノ秒単位での時間間隔指定                 |
|               | from_secs        | 秒単位での時間間隔指定                   |



## リファレンスマニュアル Python 編

| クラス                 | メソッド(空白は定数等)    | 概要                                     |
|---------------------|-----------------|----------------------------------------|
| EulerAngles         | __init__        | オイラー角の指定                               |
|                     | XYZ             | X-Y-Z オイラー角の指定                         |
|                     | ZYZ             | Z-Y-Z オイラー角の指定                         |
| FixedCompletionTime | __init__        | Fixed completion time モードの Silencer 設定 |
| FixedUpdateRate     | __init__        | Fixed update rate モードの Silencer 設定     |
| FociSTM             | __init__        | 単一焦点～8 焦点までをサポートする STM 指示              |
|                     | into_nearest    | 最近値の取得                                 |
|                     | sampling_config | サンプリング設定の取得                            |
| Focus               | __init__        | 単焦点 Gain 指示                            |
| FocusOption         | __init__        | 単焦点 Gain オプション                         |
| ForceFan            | __init__        | 強制ファン動作指示                              |
| GainGroup           | __init__        | 振動子別指定 Gain 指示                         |
| GainSTM             | __init__        | 任意 Gain をサポートする STM 指示                 |
|                     | into_nearest    | 最近値の取得                                 |
|                     | sampling_config | サンプリング設定の取得                            |
| GainSTMMode         |                 | GainSTM のデータ転送方式                       |
| GainSTMOption       | __init__        | GainSTM オプション                          |
| Geometry            | __init__        | AUTD3 デバイスの配置情報                        |
|                     | center          | 振動子の中心座標を取得                            |
|                     | num_devices     | AUTD3 デバイスの数を取得                        |
|                     | num_transducers | 全振動子の数を取得                              |
|                     | reconfigure     |                                        |
| GPIOPin             |                 | GPIO 入力ピンの番号                           |
| GPIOOut             |                 | GPIO 出力ピンの番号                           |
| GPIOOutputs         | __init__        | GPIO 出力指定指示                            |
| GPIOOutputType      |                 | GPIO 出力種別                              |
| GPIOOutputType      | BaseSignal      | 基準信号 (超音波と同じ周波数の Duty 比 50%矩形波)        |
|                     | Direct          | 指定した値を出力する                             |
|                     | ForceFan        | ForceFan フラグがアサートされているかどうか             |
|                     | IsStmMode       | FociSTM/GainSTM が使用されているかどうか           |
|                     | ModIdx          | Modulation のインデックスが指定した値のときに High になる  |
|                     | ModSegment      | Modulation のセグメント                      |
|                     | PwmOut          | 指定した振動子の PWM 出力                        |
|                     | StmIdx          | STM のインデックスが指定した値のときに High になる         |
|                     | StmSegment      | STM のセグメント                             |
|                     | Sync            | EtherCAT 同期信号                          |
|                     | SyncDiff        | システム時間の補正中に High になる                   |
|                     | SysTimeEq       | 指定したシステム時間の間 High になる                  |
|                     | Thermo          | 温度センサーがアサートされているかどうか                   |
| Group               | __init__        | AUTD3 デバイスのグループ設定                      |
| Intensity           | __init__        | 出力音圧指定                                 |
| Nop                 | __init__        |                                        |
| Null                | __init__        | ゼロ振幅 Gain 指示                           |
| OutputMask          | __init__        | 出力マスク指示                                |
|                     | with_segment    | 対象セグメント変更                              |

## リファレンスマニュアル Python 編

| クラス                   | メソッド(空白は定数等)    | 概要                                     |
|-----------------------|-----------------|----------------------------------------|
| ParallelMode          | __init__        | 並列処理モード (未サポート)                        |
| Phase                 | __init__        | 振動子毎の位相補正值                             |
|                       | radian          | 位相補正值の取得                               |
| PhaseCorrection       | __init__        | 位相補正指示                                 |
| Plane                 | __init__        | 平面波 Gain 指示                            |
| PlaneOption           | __init__        | 平面波 Gain オプション                         |
| PulseWidth            | __init__        | PWM パルス幅指定                             |
|                       | from_duty       | デューティ比で指定                              |
|                       | pulse_width     | パルス幅                                   |
| PulseWidthEncoder     | __init__        | PWM パルス幅変更指示                           |
| ReadsFPGAState        | __init__        | FPGA 状態取得指示                            |
| Remote                | __init__        | リモート接続 Link                            |
| RemoteOption          | __init__        | リモート接続 Link オプション                      |
| SamplingConfig        | __init__        | Modulation/STM のサンプリング設定               |
|                       | freq            | 周波数を取得                                 |
|                       | into_nearest    | 最近値を取得                                 |
|                       | period          | 周期を取得                                  |
| Segment               |                 | セグメント番号                                |
| SenderOption          | __init__        | 送信時設定                                  |
| Silencer              | __init__        | サイレンサ指示                                |
|                       | disable         | サイレンサ無効                                |
| Sine                  | __init__        | 正弦波変調                                  |
|                       | into_nearest    | 最近値を取得                                 |
|                       | sampling_config | サンプリング設定の取得                            |
| SineOption            | __init__        | 正弦波変調オプション                             |
| Square                | __init__        | 矩形波変調                                  |
|                       | into_nearest    | 最近値を取得                                 |
|                       | sampling_config | サンプリング設定の取得                            |
| SquareOption          | __init__        | 矩形波変調オプション                             |
| Static                | __init__        | 変調なし変調                                 |
|                       | sampling_config | サンプリング設定の取得                            |
| SwapSegmentFociSTM    | __init__        | FociSTM セグメント切り替え指示                    |
| SwapSegmentGain       | __init__        | Gain セグメント切り替え指示                       |
| SwapSegmentGainSTM    | __init__        | GainSTM セグメント切り替え指示                    |
| SwapSegmentModulation | __init__        | Modulation セグメント切り替え指示                 |
| Transducer            | __init__        | 各振動子毎の情報                               |
|                       | dev_idx         | 振動子が属するデバイスのインデックス                     |
|                       | idx             | 振動子のローカルインデックス                         |
|                       | position        | 振動子の位置                                 |
| TwinCAT               | __init__        | TwinCAT 接続 Link                        |
| Uniform               | __init__        | 全振動子への同一位相/振幅指示                        |
| WithFiniteLoop        | __init__        | Modulation 及び FociSTM/GainSTM のループ設定指示 |
| WithSegment           | __init__        | セグメント指定送信指示                            |

## ○pyautd3\_link\_ethercrab パッケージ

| クラス             | メソッド(空白は定数等) | 概要                      |
|-----------------|--------------|-------------------------|
| EtherCrab       | __init__     | EtherCrab 接続 Link       |
| EtherCrabOption | __init__     | EtherCrab 接続 Link オプション |
| Status          | __init__     | エラー発生時の状態               |

## 4.2. Controller クラス

### (1) 概要

- AUTD3 デバイスに対する操作の指示、及び、現在の状態の提供等を行う。
- このクラスは Geometry クラスを継承する。

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/controller.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/controller.html)

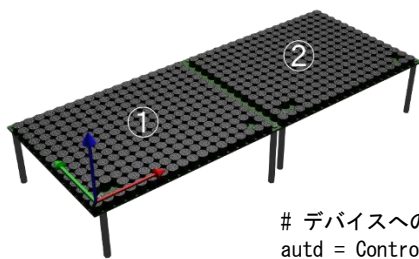
### (2) 生成

- Controller クラスのスタティックメソッド「open メソッド」を用いて生成する。

### (3) open メソッド

| 引数/戻り値:型                 | 内容                 |
|--------------------------|--------------------|
| devices: Iterable[AUTD3] | 構成する AUTD3 クラスのリスト |
| link: Link               | 使用するリンク            |
| 戻り値: Controller          | Controller インスタンス  |

- AUTD3 デバイスとの通信を開始し、戻り値として Controller インスタンスを返す。
- 下記の様に 2 つの AUTD3 デバイスを横に並べて設置し、EtherCrab リンクを用いる場合の引数は下記のようになる。



```
# デバイスへの接続(EtherCrab を使用)
autd = Controller.open(
    [ # AUTD3 ボードの指定
      AUTD3(pos=[0.0, 0.0, 0.0], rot=[1, 0, 0, 0]), # 1 枚目のボードの位置と向き
      AUTD3(pos=[AUTD3.DEVICE_WIDTH, 0.0, 0.0], rot=[1, 0, 0, 0]) # 2 枚目
    ],
    EtherCrab(
        err_handler=err_handler, # エラー発生時の処理関数指定
        option=EtherCrabOption() # デフォルトオプション
    )
)
```

### (4) close メソッド

| 引数/戻り値:型 | 内容 |
|----------|----|
| 引数なし     |    |
| 戻り値なし    |    |

- AUTD3 デバイスとの通信を終了する。

## (5) send メソッド

| 引数/戻り値:型                                | 内容           |
|-----------------------------------------|--------------|
| d: Datagram   tuple[Datagram, Datagram] | 送信するデータ (指示) |
| 戻り値                                     | なし           |

- AUTD3 デバイスにデータ(各種指示)を送信する。

- 引数には下記のデータ(クラス)を指定する。

| 送信できるデータ(クラス)         | 内容                                     |
|-----------------------|----------------------------------------|
| Bessel                | ベッセルビーム Gain 指示                        |
| Custom                | 自由な音場 Gain 指示                          |
| FociSTM               | 単一焦点～8 焦点までをサポートする STM 指示              |
| Focus                 | 単焦点 Gain 指示                            |
| ForceFan              | 強制ファン動作指示                              |
| GainSTM               | 任意 Gain をサポートする STM 指示                 |
| GPIOOutputs           | GPIO 出力指定指示                            |
| Group                 | AUTD3 デバイスのグループ設定                      |
| Null                  | ゼロ振幅 Gain 指示                           |
| OutputMask            | 出力マスク指示                                |
| PhaseCorrection       | 位相補正指示                                 |
| Plane                 | 平面波 Gain 指示                            |
| PulseWidthEncoder     | PWM パルス幅変更指示                           |
| ReadsFPGAState        | FPGA 状態取得指示                            |
| Silencer              | サイレンサ指示                                |
| SwapSegmentFociSTM    | FociSTM セグメント切り替え指示                    |
| SwapSegmentGain       | Gain セグメント切り替え指示                       |
| SwapSegmentGainSTM    | GainSTM セグメント切り替え指示                    |
| SwapSegmentModulation | Modulation セグメント切り替え指示                 |
| Uniform               | 全振動子への同一位相/振幅指示                        |
| WithFiniteLoop        | Modulation 及び FociSTM/GainSTM のループ設定指示 |
| WithSegment           | セグメント指定送信指示                            |

- 2 要素タプルにすることにより、二つのデータを同時に送信することができる。

```

g = Focus( # 単一焦点の位相制御
    pos=autd.center() + np.array( [0.0, 0.0, 150.0] ), # 中心から上に 150mm の位置
    option=FocusOption(), # デフォルトオプション
)
m = Sine( # Sin 振幅
    freq=150 * Hz, # 150Hz
    option=SineOption(), # デフォルトオプション
)
autd.send( (m, g) ) # 位相制御と振幅制御を同時に送信

```

## (6) firmware\_version メソッド

| 引数/戻り値:型                | 内容                    |
|-------------------------|-----------------------|
| 引数なし                    |                       |
| 戻り値: list[FirmwareInfo] | AUTD3 デバイス毎のファームウェア情報 |

- ・ 戻り値として AUTD3 デバイスのファームウェアバージョンのリストを返す。

## (7) geometry メソッド

| 引数/戻り値:型      | 内容          |
|---------------|-------------|
| 引数なし          |             |
| 戻り値: geometry | geometry 情報 |

- ・ 戻り値として現在 AUTD3 デバイス情報(geometry クラス)を返す。

## (8) link メソッド

| 引数/戻り値:型  | 内容              |
|-----------|-----------------|
| 引数なし      |                 |
| 戻り値: link | 現在の link インスタンス |

- ・ 通信で使用している Link インスタンスを返す。

## (9) sender メソッド

| 引数/戻り値:型             | 内容            |
|----------------------|---------------|
| option: SenderOption | 送信設定          |
| 戻り値 : Sender         | Sender インスタンス |

- ・ 指定された送信設定が適用された送信管理を返す。

- ・ 送信設定(SenderOption)には、下記を指定する。

| 引数:型                       | 内容       |
|----------------------------|----------|
| send_interval: Duration    | 送信間隔     |
| receive_interval: Duration | 受信間隔     |
| timeout: Duration          | タイムアウト時間 |

## (10) center メソッド/num\_deveices メソッド/num\_transducers メソッド

- ・ 現在のデバイス状態を返す。詳細は Geometry クラスの同名メソッドを参照。

## (11) fpga\_state メソッド

| 引数/戻り値:型              | 内容            |
|-----------------------|---------------|
| なし                    |               |
| 戻り値 : list[FPGAState] | Sender インスタンス |

- AUTD3 デバイス(FPGA)の状態を取得する。
- このメソッドを呼び出す前に、ReadsFPGAState を送信し、状態取得を有効化しておく必要がある。

```
autd.send( ReadFPGAState(lambda _: True) )  
info = autd.fpga_state()
```

### 4.3. AUTD3 クラス

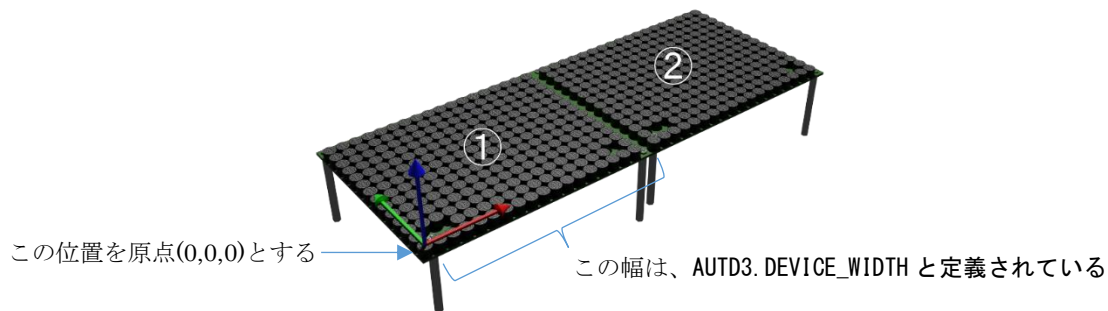
#### (1) 概要

- ・ 接続されている個々の AUTD3 デバイス情報を管理する。
- ・ AUTD3 デバイスの物理的な設置状態に合わせて定義し、接続時に指定する。

#### (2) コンストラクタ

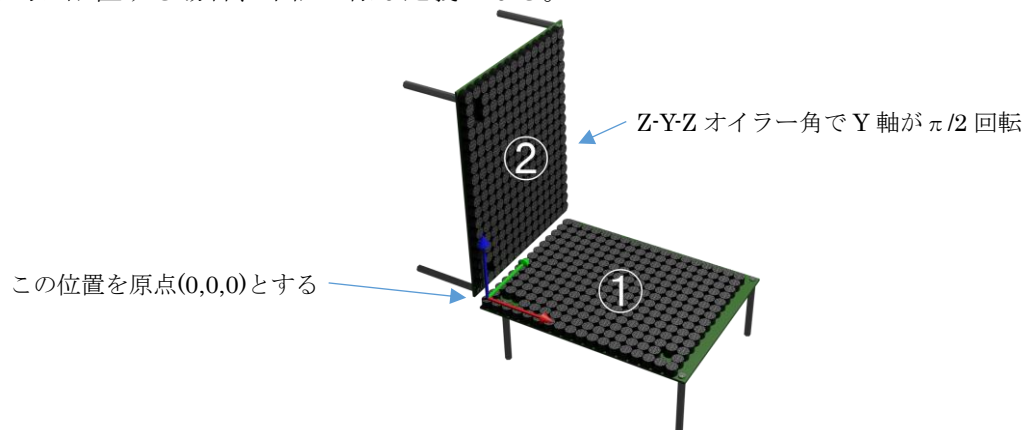
| 引数:型           | 内容                                    |
|----------------|---------------------------------------|
| pos: ArrayLike | AUTD3 ボードの位置 (座標を [x, y, z] で指定)      |
| rot: ArrayLike | AUTD3 ボードの向き (ベクトルを [x, y, z, w] で指定) |

- ・ 横に 2 台並べて設置する場合は、下記の様な定義となる。



- ① `AUTD3( pos=[0.0, 0.0, 0.0 ], rot=[1, 0, 0, 0] )`
- ② `AUTD3( pos=[AUTD3.DEVICE_WIDTH, 0.0, 0.0 ], rot=[1, 0, 0, 0] )`

- ・ 下記の様に立体的に配置する場合、下記の様な定義となる。



- ① `AUTD3( pos=[0.0, 0.0, 0.0 ], rot=[1, 0, 0, 0] )`
- ② `AUTD3( pos=[0.0, 0.0, AUTD3.DEVICE_WIDTH ],  
rot= EulerAngles.ZYZ(0 * rad, np.pi / 2 * rad, 0 * rad) )`

- ・ Controller クラスの open メソッドの第 1 引数に上記のリストを指定する。

参考 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/tutorial/multiple.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/tutorial/multiple.html)

#### 4.4. Geometry クラス/Device クラス/Transducer クラス

##### (1) 概要

- AUTD3 デバイスを構成する要素の状態等について管理する。  
これらのクラスはユーザプログラムで生成することではなく、現在の状態を Controller クラスが提供する。
- Controller クラスは Geometry クラスを継承している。

##### (2) center メソッド (Geometry クラス)

| 引数/戻り値:型           | 内容             |
|--------------------|----------------|
| 引数なし               |                |
| 戻り値: numpy.ndarray | 中心座標 [x, y, z] |

- 戻り値として空間(全 AUTD3 デバイス)の中心位置を返す。

##### (3) num\_devices メソッド (Geometry クラス)

| 引数/戻り値:型 | 内容           |
|----------|--------------|
| 引数なし     |              |
| 戻り値: int | AUTD3 デバイスの数 |

- 戻り値として AUTD3 デバイスの数を返す。

##### (4) num\_transducers メソッド (Geometry クラス)

| 引数/戻り値:型 | 内容    |
|----------|-------|
| 引数なし     |       |
| 戻り値: int | 振動子の数 |

- 戻り値として全振動子の数を返す。

##### (5) axial\_direction メソッド (Device クラス)

| 引数/戻り値:型           | 内容           |
|--------------------|--------------|
| 引数なし               |              |
| 戻り値: numpy.ndarray | デバイスの軸方向ベクトル |

- デバイスの軸方向ベクトル (振動子が向く方向)を返す。



## (6) center メソッド (Device クラス)

| 引数/戻り値:型                        | 内容   |
|---------------------------------|------|
| 引数なし                            |      |
| 戻り値: <code>numpy.ndarray</code> | 中心座標 |

- ・ 戻り値として、デバイスの中心位置を返す。  
(Geometry の center メソッドは、全てのデバイスを合わせた空間の中心位置を返すが、Device の center メソッドは、各デバイスの中心位置を返す)

## (7) idx メソッド (Device クラス)

| 引数/戻り値:型              | 内容   |
|-----------------------|------|
| 引数なし                  |      |
| 戻り値: <code>int</code> | 中心座標 |

- ・ 戻り値として、デバイスのインデックスを表す数値を返す。

## (8) num\_transducers メソッド (Device クラス)

| 引数/戻り値:型              | 内容       |
|-----------------------|----------|
| 引数なし                  |          |
| 戻り値: <code>int</code> | 振動子の数を取得 |

- ・ 戻り値として、デバイス内の振動子数を返す。

## (9) rotation メソッド (Device クラス)

| 引数/戻り値:型                        | 内容     |
|---------------------------------|--------|
| 引数なし                            |        |
| 戻り値: <code>numpy.ndarray</code> | 回転ベクトル |

- ・ 戻り値として、デバイスの回転ベクトルを返す。

## (10) x\_direction メソッド/ y\_direction メソッド (Device クラス)

| 引数/戻り値:型                        | 内容             |
|---------------------------------|----------------|
| 引数なし                            |                |
| 戻り値: <code>numpy.ndarray</code> | デバイスの X 方向ベクトル |

- ・ 戻り値として、デバイスの X 方向ベクトル、又は、Y 方向ベクトルを返す。

## (11) dev\_idx メソッド (Transducer クラス)

| 引数/戻り値:型 | 内容              |
|----------|-----------------|
| 引数なし     |                 |
| 戻り値:int  | 所属するデバイスのインデックス |

- ・ 戻り値として、振動子が所属するデバイスのインデックスを返す。

## (12) idx メソッド (Transducer クラス)

| 引数/戻り値:型 | 内容         |
|----------|------------|
| 引数なし     |            |
| 戻り値:int  | 振動子のインデックス |

- ・ 戻り値として、デバイス内の振動子インデックス値を返す。

## (13) position メソッド (Transducer クラス)

| 引数/戻り値:型           | 内容     |
|--------------------|--------|
| 引数なし               |        |
| 戻り値:numpy. ndarray | 振動子の位置 |

- ・ 戻り値として、振動子の位置を返す。

#### 4.5. TwinCAT クラス

##### (1) 概要

- TwinCAT を用いて AUTD3 デバイスと通信を行う
- Link クラスを継承する。(Controller の open メソッドにて指定する)

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/link/twincat.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/link/twincat.html)

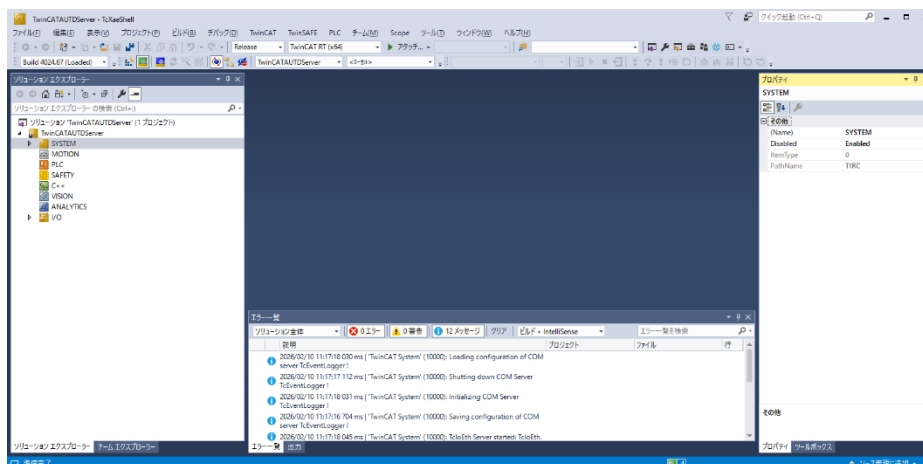
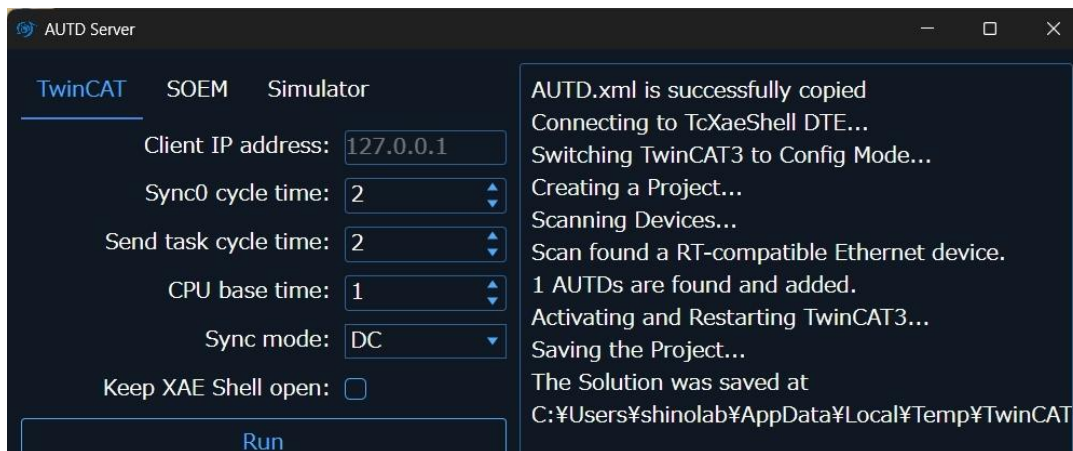
##### (2) コンストラクタ

- 引数無しで TwinCAT 用の Link オブジェクトを生成する。

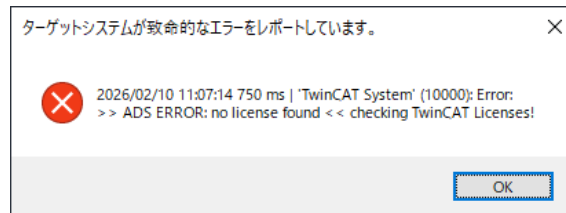
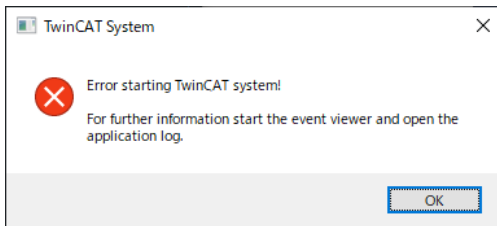
##### (3) AUTD3 Server

- TwinCAT リンクを使用する場合、事前に AUTD3 Server を起動し、Run ボタン押下で接続可能な状態にしておく必要がある。

AUTD3 デバイスが見つかった場合、サーバー画面には下記の様なメッセージ (x AUTDs are found and added.)が表示され、TcXaeShell 画面(Visual studio)が表示される。

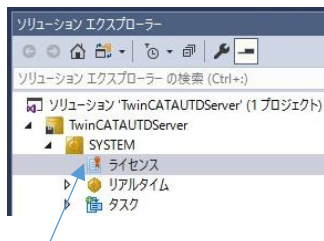


- TwinCAT のライセンスが切れている場合、下記の様なダイアログが表示される。

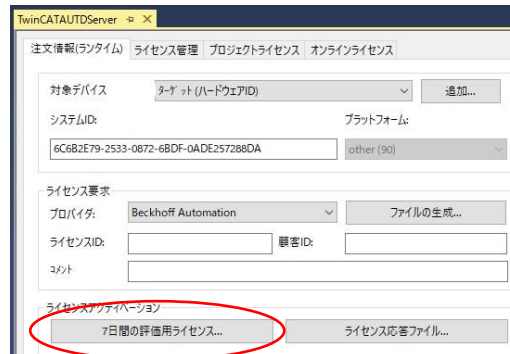


- 下記の手順で、7日間限定のトライアルライセンスを発行し使用することができる。

- ① TcXaeShell(Visual Studio)ウィンドウの「ソリューションエクスプローラ」から「TwinCATAUTDServer」－「SYSTEM」を展開し、「ライセンス」をダブルクリックして「TwinCATAUTDServer」のダイアログを表示させる。



「ライセンス」をダブルクリックすると  
右の画面が表示される。



- ② 「7日間の評価用ライセンス...」をクリックし「セキュリティコードの入力」画面が表示させ、画面に表示された文字を入力する。
- ③ TcXaeShell(Visual Studio)ウィンドウを右上の×ボタンで閉じて、AUTD3 サーバー画面の Run ボタンを再度クリックする。

#### 4.6. EtherCrab クラス/EtherCrabOption クラス

##### (1) 概要

- EtherCrab を用いて AUTD3 デバイスと通信を行う。
- EtherCrab クラスは Link クラスを継承する。(Controller の open メソッドにて指定する)

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/link/ethercrab.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/link/ethercrab.html)

##### (2) コンストラクタ

###### ○EtherCrab クラス

| 引数:型                                                  | 内容      |
|-------------------------------------------------------|---------|
| err_handler:<br>Callable[<br>[int, Status],<br>None ] | エラーハンドラ |
| option: EtherCrabOption                               | 接続オプション |

###### ○EtherCrabOption クラス

| 引数:型                            | 内容                                           |
|---------------------------------|----------------------------------------------|
| ifname: str                     | ネットワークインターフェース名                              |
| state_check_period:<br>Duration | エラーが出ているかどうかを確認する間隔                          |
| sync0_period: Duration          | 同期信号の周期                                      |
| sync_tolerance: Duration        | 同期許容レベル<br>初期化時、各デバイスのシステム時間差がこの値以下になるまで待機する |
| sync_timeout: Duration          | 同期タイムアウト値                                    |

- EtherCrab 用の Link オブジェクトを生成する。
- EtherCrabOption クラスは、EtherCrab クラスのインスタンス生成時に指定する。

- EtherCrab を Windows 上で使用する場合、  
Npcap を「WinPcap API compatible Mode」でインストールしておく必要がある。

Npcap : <https://npcap.com/>

- EtherCrab を Linux 上で使用する場合、  
下記コマンドにて python に NIC の使用権限を付与しておく必要がある。

`sudo setcap cap_net_admin,cap_net_raw=eip python` のパス

例) Ubuntu 24.04 の場合

`sudo setcap cap_net_admin,cap_net_raw=eip /usr/bin/python3.12`

- EtherCrab を MacOS で使用する場合、  
下記コマンドにて管理者権限を付与しておく必要がある。

`sudo chmod +r /dev/bpf*`

### (3) Status クラス

- EtherCrab リンクの処理で異常等が発生した場合、  
エラー情報が格納された **Status** を引数として、接続時に指定されたエラーハンドラを呼び出す。

#### 4.7. Remote クラス/RemoteOption クラス

##### (1) 概要

- ・ リモートサーバ経由で AUTD3 デバイスと通信を行う。
- ・ Remote クラスは Link クラスを継承する。(Controller の open メソッドにて指定する)

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/link/remote.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/link/remote.html)

##### (2) コンストラクタ

###### ○Remote クラス

| 引数:型                 | 内容          |
|----------------------|-------------|
| addr: str            | 接続先 IP アドレス |
| option: RemoteOption | 接続オプション     |

###### ○RemoteOption クラス

| 引数:型              | 内容      |
|-------------------|---------|
| timeout: Duration | タイムアウト値 |

- ・ EtherCrab 用の Link オブジェクトを生成する。
- ・ EtherCrabOption クラスは、EtherCrab クラスのインスタンス生成時に指定する。

#### 4.8. Silencer クラス

##### (1) 概要

- 振幅変調に伴う可聴音の発生を抑える Silencer 機能の動作を指示する。

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/silencer.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/silencer.html)

##### (2) コンストラクタ

| 引数:型                                                                               | 内容                                  |
|------------------------------------------------------------------------------------|-------------------------------------|
| config: T<br>T: (FixedCompletionSteps,<br>FixedCompletionTime,<br>FixedUpdateRate) | モード指定。<br>省略時は FixedCompletionSteps |

- 引数には以下の何れかを指定できる。

| Silencer モード指定                                                                                    | 内容                               |
|---------------------------------------------------------------------------------------------------|----------------------------------|
| FixedCompletionTime(<br>intensity : c_uint16,<br>phase : c_uint16,<br>strict : ctypes.c_bool<br>) | 位相/振幅変化が完了するまでの時間がすべての振動子で同一のモード |
| FixedUpdateRate(<br>intensity : c_uint16,<br>phase : c_uint16<br>)                                | 位相/振幅変化速度がすべての振動子で同一のモード         |

##### (3) disable メソッド(@staticmethod)

- Silencer 動作を無効にする。

```
autd = Controller.open( 略 )
```

```
autd.send( Silencer() )           # Silencer 有効
```

```
autd.send( Silencer.disable() )   # Silencer 無効
```



#### 4.9. Focus クラス/FocusOption クラス

##### (1) 概要

- 単一焦点を生成する。  
(空間上の一点に触覚を発生させる)
- Gain クラスを継承する。

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/gain/focus.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/gain/focus.html)

##### (2) コンストラクタ

###### ○Focus クラス

| 引数:型                | 内容                   |
|---------------------|----------------------|
| pos: ArrayLike      | 焦点を発生させる座標 [x, y, z] |
| option: FocusOption | オプション                |

###### ○FocusOption クラス

| 引数:型                 | 内容      |
|----------------------|---------|
| intensity: Intensity | 出力振幅    |
| phase_offset: Phase  | 位相オフセット |

#### 4.10. Plane クラス/PlaneOption クラス

##### (1) 概要

- 平面波を出力させる。  
(一点ではなく全体に触覚を発生させる)
- Gain クラスを継承する。

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/gain/plane.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/gain/plane.html)

##### (2) コンストラクタ

###### ○Focus クラス

| 引数:型                 | 内容             |
|----------------------|----------------|
| direction: ArrayLike | 平面波の方向(法線ベクトル) |
| option: PlaneOption  | オプション          |

###### ○PlaneOption クラス

| 引数:型                 | 内容      |
|----------------------|---------|
| intensity: Intensity | 出力振幅    |
| phase_offset: Phase  | 位相オフセット |

#### 4.11. Bessel クラス/BesselOption クラス

##### (1) 概要

- ベッセルビームを生成する。  
(Focus の様な一点ではなく、直線状の触覚を発生させる)
- Gain クラスを継承する。

参照 : <https://shinolab.github.io/autd3-doc/jp/Users Manual/API/gain/bessel.html>

##### (2) コンストラクタ

###### ○Bessel クラス

| 引数:型                 | 内容                   |
|----------------------|----------------------|
| apex: ArrayLike      | ビーム仮想円錐の頂点 (振動子面の位置) |
| direction: ArrayLike | ビームの方向 (触覚を有無直線の向き)  |
| option: PlaneOption  | オプション                |

###### ○BesselOption クラス

| 引数:型                 | 内容      |
|----------------------|---------|
| intensity: Intensity | 出力振幅    |
| phase_offset: Phase  | 位相オフセット |

#### 4.12. FociSTM クラス

##### (1) 概要

- 1～8 個の単一焦点を生成する時空間変調(STM)を指示する。
- Focus 相当の単一焦点の座標リストをあらかじめ指定しておき、指定した周波数で順次焦点を発生させる。(リストで指定した座標での焦点発生を繰り返す)

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/stm.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/stm.html)  
[https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/stm/focus.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/stm/focus.html)

##### (2) コンストラクタ

###### ○FociSTM クラス

| 引数:型                                                                                         | 内容            |
|----------------------------------------------------------------------------------------------|---------------|
| foci: (<br>Iterable[ArrayLike]  <br>Iterable[ControlPoint]  <br>Iterable[ControlPoints]<br>) | 焦点のリスト        |
| config:<br>SamplingConfig  <br>Freq[float]   Duration                                        | 周期またはサンプリング設定 |

- 複数の焦点を発生する場合は、ControlPoints で各焦点毎の焦点リストを指定する。

#### 4.13. GainSTM クラス/GainSTMOption クラス

##### (1) 概要

- 任意の Gain を扱う時空間変調(STM)を指示する。
- 単一焦点(Focus)だけでなく様々な Gain(Bessel や Plane 等)を一定時間毎に繰り返す触感を発生させる。

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/stm.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/stm.html)  
[https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/stm/gain.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/stm/gain.html)

##### (2) コンストラクタ

###### ○GainSTM クラス

| 引数:型                                                                                         | 内容            |
|----------------------------------------------------------------------------------------------|---------------|
| gains: Iterable[Gain]                                                                        | Gain のリスト     |
| config:<br>SamplingConfig  <br>Freq[float]  <br>Duration  <br>FreqNearest  <br>PeriodNearest | 周期またはサンプリング設定 |
| option: GainSTMOption                                                                        | オプション         |

###### ○GainSTMOption クラス

| 引数:型              | 内容            |
|-------------------|---------------|
| mode: GainSTMMode | GainSTM モード指定 |

- GainSTM では位相と振幅のデータを事前に送っておく必要があり、送信に時間がかかることで遅延が発生する可能性がある。  
この為、データ送信時間を短縮させるための GainSTM モードを指定できる。
- GainSTM モードには、下記の何れかを指定できる。

| GainSTM モード        | 内容               |
|--------------------|------------------|
| PhaseIntensityFull | 振幅/位相データをすべて送信   |
| PhaseFull          | 位相データのみを送信       |
| PhaseHalf          | 位相を 4bit に圧縮して送信 |

#### 4.14. Null クラス

##### (1) 概要

- ・ 振幅を発生させない。
- ・ Gain クラスを継承する。
- ・ Null0を送信すると振幅が発生しなくなるため、出力が停止した状態となる。

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/stm.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/stm.html)  
[https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/stm/gain.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/stm/gain.html)

##### (2) コンストラクタ

###### ○Null クラス

| 引数:型 | 内容 |
|------|----|
|      | なし |

## 4.15. Sine クラス/SineOption クラス

## (1) 概要

- 正弦波の AM 変調を指示する。

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/modulation.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/modulation.html)  
[https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/modulation/sine.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/modulation/sine.html)

## (2) コンストラクタ

## ○Sine クラス

| 引数:型               | 内容    |
|--------------------|-------|
| freq: Freq[T]      | 周波数   |
| option: SineOption | オプション |

## ○SineOption クラス

| 引数:型                            | 内容                                                    |
|---------------------------------|-------------------------------------------------------|
| intensity: int                  | 振幅                                                    |
| offset: int                     | オフセット (DC 成分)                                         |
| phase: Angle                    | 位相オフセット                                               |
| clamp: bool                     | true 時、波形が [0, 255] 範囲になる様に補正される。<br>false 時は、エラーとなる。 |
| sampling_config: SamplingConfig | サンプリング設定                                              |

- 出力不可能な周波数を指定した場合、実行時にエラーが発生する。

into\_nearest() メソッドを使用すると出力可能な周波数に最も近い周波数に補正することができる。

```
Sine(freq=150.0 * Hz, option=SineOption()).into_nearest()
```

- 出力波形は下記波形となる。

$$[ \text{intensity} / 2 \times \sin( 2\pi \times \text{freq} \times t + \text{phase} ) + \text{offset} ]$$

#### 4.16. Square クラス/SquareOption クラス

##### (1) 概要

- 矩形波の AM 変調を指示する。

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/modulation.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/modulation.html)  
[https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/modulation/square.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/modulation/square.html)

##### (2) コンストラクタ

###### ○Square クラス

| 引数:型                 | 内容    |
|----------------------|-------|
| freq: Freq[T]        | 周波数   |
| option: SquareOption | オプション |

###### ○SquareOption クラス

| 引数:型                            | 内容       |
|---------------------------------|----------|
| low : int                       | 低レベル値    |
| high : int                      | 高レベル値    |
| duty : float                    | デューティー比  |
| sampling_config: SamplingConfig | サンプリング設定 |

- 出力不可能な周波数を指定した場合、実行時にエラーが発生する。  
into\_nearest()メソッドを使用すると出力可能な周波数に最も近い周波数に補正することができる。

```
Square (freq=150.0 * Hz, option=Square ()).into_nearest()
```



#### 4.17. Static クラス

##### (1) 概要

- ・ AM 変調を行わないことを指示する。
- ・ AM 変調を行わない場合、焦点位置等の変化がないと触感が発生しない。

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/modulation.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/modulation.html)  
[https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/modulation/static.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/modulation/static.html)

##### (2) コンストラクタ

| 引数:型           | 内容 |
|----------------|----|
| intensity: int | 振幅 |

## 4.18. EtherCrab クラス/EtherCrabOption クラス

## (1) 概要

- ・ オープンソースの EtherCAT Master ライブラリである「EtherCrab」を用いたリンクを提供する。
- ・ Windows 上から使用する場合、  
npcap をインストール時に「WinPcap API compatible mode」を指定してインストールする必要がある。

参照 : [https://shinolab.github.io/autd3-doc/jp/Users\\_Manual/API/link/ethercrab.html](https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/link/ethercrab.html)

## (2) コンストラクタ

## ○EtherCrab クラス

| 引数:型                                          | 内容              |
|-----------------------------------------------|-----------------|
| err_handler:<br>Callable[[int, Status], None] | エラー発生時のコールバック関数 |
| option: EtherCrabOption                       | オプション           |

## ○EtherCrabOption クラス

| 引数:型                         | 内容                                                                     |
|------------------------------|------------------------------------------------------------------------|
| ifname: str                  | ネットワークインターフェース名。None の場合、自動的に選択される。                                    |
| state_check_period: Duration | エラー発生を確認する間隔。                                                          |
| sync0_period: Duration       | 同期信号の周期。<br>大量のデバイスを接続し動作が不安定になった場合はこの値を増やすことで安定するが、エラーとにならない最小値が望ましい。 |
| sync_tolerance: Duration     | 同期許容レベル。<br>初期化時に各デバイスのシステム時間差がこの値以下になるまで待機する。<br>(変更は推奨されない)          |
| sync_timeout: Duration       | 同期タイムアウト。<br>初期化時に各デバイスのシステム時間差を測定する際にこの時間を超過するとエラーとなる。                |

- ・ エラー発生時のコールバック関数を定義し、EtherCrab の第 1 引数に指定する。  

```
def err_handler(idx: int, status: Status) -> None:
    print(f"Device[{idx}]: {status}")
```
- ・ Linux ではネットワークインターフェースの RAW パケット使用権限がない場合、デバイス検出に失敗する。  
(root 権限で実行するか、下記コマンドの様に Python 自体に許可を与える必要がある。

```
sudo setcap cap_net_admin,cap_net_raw=eip /usr/bin/python3.12
```

以上